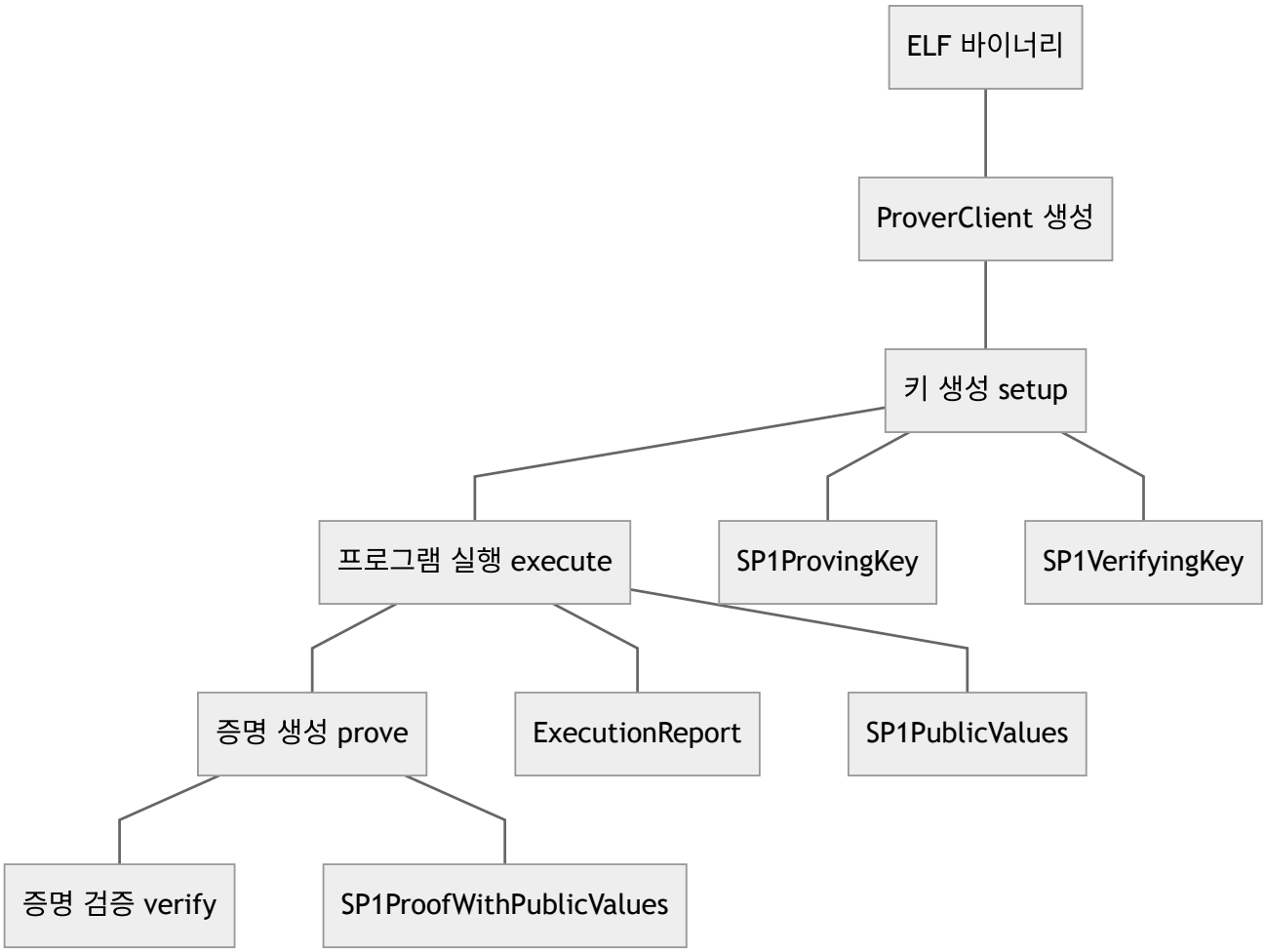
03. SP1 SDK 컴포넌트 분석

개요

SP1 SDK(Software Development Kit)는 개발자가 SP1 영지식 가상 머신을 사용하기 위한 고수준 인터페이스를 제공하는 핵심 라이브러리입니다. SDK는 다양한 증명 생성 방식(CPU, CUDA, 네트워크)을 지원하며, 통일된 API를 통해 프로그램 실행부터 증명 생성 및 검증까지의 전체 워크플로우를 관리합니다.

전체 워크플로우 개요

SP1 SDK의 전체 워크플로우는 다음과 같은 단계로 구성됩니다:



핵심 컴포넌트 구조

1. ProverClient - 통합 클라이언트 인터페이스

`ProverClient` 는 SP1 SDK의 진입점 역할을 하는 팩토리 클래스입니다.

```
pub struct ProverClient;


impl ProverClient {
// 환경 변수에서 설정을 로드하는 EnvProver 생성
pub fn from_env() -> EnvProver {
```

```
EnvProver::new()
}
// 빌더 패턴을 통한 프로버 구성
pub fn builder() -> ProverClientBuilder {
ProverClientBuilder
}
}
```

주요 특징:

- 빌더 패턴을 통한 유연한 프로버 구성
- 환경 변수 기반 자동 설정 지원
- 다양한 프로버 타입에 대한 통일된 인터페이스 제공

사용 예제:

```
use sp1_sdk::{ProverClient, SP1Stdin};

// CPU 프로버 생성
let client = ProverClient::builder().cpu().build();

// 환경 변수 기반 프로버 생성
std::env::set_var("SP1_PROVER", "cuda");
let client = ProverClient::from_env();

// 프로그램 실행 및 증명 생성
```

```
let elf = include_bytes!("../program.elf");
let stdin = SP1Stdin::new();
let (pk, vk) = client.setup(elf);
let proof = client.prove(&pk, &stdin).run().unwrap();
```

2. Prover 트레이트 - 공통 인터페이스

모든 프로버 구현체가 따라야 하는 핵심 트레이트입니다.

```
pub trait Prover<C: SP1ProverComponents>: Send + Sync {
fn inner(&self) -> &SP1Prover<C>;
fn version(&self) -> &str;
fn setup(&self, elf: &[u8]) -> (SP1 provingKey, SP1VerifyingKey);
fn execute(&self, elf: &[u8], stdin: &SP1Stdin) -> Result<(SP1PublicValues, Exe
fn prove(&self, pk: &SP1ProvingKey, stdin: &SP1Stdin, mode: SP1ProofMode) -> Re
fn verify(&self, bundle: &SP1ProofWithPublicValues, vkey: &SP1VerifyingKey) ->
}
```

키 생성 과정 (setup) 상세 분석

`setup()` 메서드는 ELF 바이너리를 분석하여 증명 및 검증 키를 생성하는 복잡한 과정입니다:

```
impl<C: SP1ProverComponents> Prover<C> for CpuProver {
fn setup(&self, elf: &[u8]) -> (SP1ProvingKey, SP1VerifyingKey) {
// 1. ELF 파싱 및 프로그램 추출
```

```
let program = Program::from(elf).expect("Failed to parse ELF");

// 2. 프로그램 분석 및 최적화

let optimized_program = self.optimize_program(program);

// 3. 회로 키 생성

let (pk, vk, _, _) = self.inner().setup(&optimized_program);

// 4. 키 검증 및 메타데이터 추가

let proving_key = SP1ProvingKey {

pk,

elf: elf.to_vec(),

vk_digest: vk.hash(),

};

let verifying_key = SP1VerifyingKey {

vk,

elf_digest: sha256(elf),

};

(proving_key, verifying_key)

}

}
```

키 생성 단계별 세부 과정:

1. **ELF 분석**: 바이너리 구조 파싱, 심볼 테이블 추출, 메모리 레이아웃 결정
2. **프로그램 최적화**: 불필요한 명령어 제거, 루프 최적화, 메모리 접근 패턴 분석
3. **회로 구성**: 필요한 칩들 결정, 상호작용 그래프 생성, 제약 조건 컴파일

4. **키 생성**: 신뢰할 수 있는 설정 실행, 다항식 커밋 키 생성

프로그램 실행 과정 (execute) 상세 분석

```
impl<C: SP1ProverComponents> Prover<C> for CpuProver {
fn execute(&self, elf: &[u8], stdin: &SP1Stdin) -> Result<(SP1PublicValues, Exe
// 1. 실행 환경 초기화
let program = Program::from(elf)?;
let mut executor = Executor::new(program, SP1CoreOpts::default());
// 2. 입력 데이터 설정
executor.write_stdin(stdin);
// 3. 프로그램 실행
executor.run()?;
// 4. 결과 추출
let public_values = executor.get_public_values();
let report = executor.get_execution_report();
Ok((public_values, report))
}
}
```

실행 과정의 세부 단계:

1. **메모리 초기화**: 스택, 힙, 전역 변수 영역 설정
2. **레지스터 초기화**: PC, SP, 기타 시스템 레지스터 설정
3. **입력 처리**: stdin 데이터를 메모리에 로드

4. **명령어 실행**: RISC-V 명령어 순차 실행

5. **상태 추적**: 실행 통계, 메모리 사용량, 사이클 수 기록

증명 생성 과정 (prove) 상세 분석

증명 생성은 SP1의 가장 복잡한 과정으로, 여러 단계의 변환을 거칩니다:

```
impl<C: SP1ProverComponents> Prover<C> for CpuProver {
fn prove(&self, pk: &SP1ProvingKey, stdin: &SP1Stdin, mode: SP1ProofMode) -> Re
match mode {
SP1ProofMode::Core => self.prove_core(pk, stdin),
SP1ProofMode::Compressed => self.prove_compressed(pk, stdin),
SP1ProofMode::Plonk => self.prove_plonk(pk, stdin),
SP1ProofMode::Groth16 => self.prove_groth16(pk, stdin),
}
}
fn prove_core(&self, pk: &SP1ProvingKey, stdin: &SP1Stdin) -> Result<SP1ProofWi
// 1. 프로그램 실행 및 추적 생성
let (public_values, execution_record) = self.execute_and_trace(&pk.elf, stdin)?;
// 2. 샤드 분할
let shards = self.split_into_shards(execution_record)?;
// 3. 각 샤드에 대한 STARK 증명 생성
let shard_proofs = shards.par_iter()
.map(|shard| self.prove_shard(pk, shard))
.collect::<Result<Vec<_>>>()?;
```

```
// 4. 증명 메타데이터 생성
let proof = SP1ProofWithPublicValues::new(
SP1Proof::Core(shard_proofs),
public_values,
self.version().to_string(),
);
Ok(proof)
}
}
```

증명 모드별 상세 프로세스:

Core 증명 모드

- **목적**: 각 샤드에 대한 개별 STARK 증명 생성
- **특징**: 가장 빠르지만 증명 크기가 큼
- **과정**: 실행 추적 → 샤드 분할 → 개별 STARK 증명

Compressed 증명 모드

- **목적**: 여러 Core 증명을 하나로 압축
- **특징**: 중간 크기의 증명, 적당한 검증 시간
- **과정**: Core 증명 → 재귀적 압축 → 단일 증명

PLONK 증명 모드

- **목적**: 범용 검증이 가능한 SNARK 증명 생성
- **특징**: 작은 크기, 빠른 검증, 이더리움 호환
- **과정**: Compressed 증명 → PLONK 회로 → BN254 증명

Groth16 증명 모드

- **목적**: 가장 작은 크기의 증명 생성
- **특징**: 고정 크기 (192바이트), 매우 빠른 검증
- **과정**: Compressed 증명 → Groth16 회로 → 최종 증명

증명 검증 과정 (verify) 상세 분석

```
impl<C: SP1ProverComponents> Prover<C> for CpuProver {
fn  verify(&self, bundle: &SP1ProofWithPublicValues, vkey: &SP1VerifyingKey) ->
// 1. 버전 호환성 검사
if  bundle.sp1_version != self.version() {
return Err(SP1VerificationError::VersionMismatch(bundle.sp1_version.clone()));
}
// 2. 공개 값 해시 검증
self.verify_public_values_hash(bundle)?;
// 3. 증명 타입별 검증
match &bundle.proof {
SP1Proof::Core(proofs) => self.verify_core_proofs(proofs, vkey),
SP1Proof::Compressed(proof) => self.verify_compressed_proof(proof, vkey),
SP1Proof::Plonk(proof) => self.verify_plonk_proof(proof, vkey, &bundle.public_va
SP1Proof::Groth16(proof) => self.verify_groth16_proof(proof, vkey, &bundle.publi
}
}
```

```
}
```

검증 과정의 핵심 단계:

1. **메타데이터 검증**: 버전, 키 호환성, 공개 값 무결성
2. **암호학적 검증**: 증명의 수학적 정확성 확인
3. **일관성 검증**: 공개 값과 증명 내용의 일치성 확인

3. CpuProver - CPU 기반 증명 생성기

CPU를 사용하여 증명을 생성하는 기본 프로버입니다.

구조 및 특징

```
pub struct CpuProver {
pub(crate) prover: SP1Prover<CpuProverComponents>,
pub(crate) mock: bool,
}

impl CpuProver {
pub fn new() -> Self { /* ... */ }
pub fn mock() -> Self { /* ... */ }
// 실행 빌더 생성
pub fn execute<'a>(&'a self, elf: &'a [u8], stdin: &SP1Stdin) -> CpuExecuteBui
// 증명 빌더 생성
pub fn prove<'a>(&'a self, pk: &'a SP1ProvingKey, stdin: &SP1Stdin) -> CpuProv
```

```
}
```

빌더 패턴 상세 분석

CpuExecuteBuilder - 실행 설정 빌더

```
pub struct CpuExecuteBuilder<'a> {
prover: &'a SP1Prover<CpuProverComponents>,
elf: &'a [u8],
stdin: SP1Stdin,
context_builder: SP1ContextBuilder,
}

impl<'a> CpuExecuteBuilder<'a> {
/// 최대 사이클 수 설정
pub fn cycle_limit(mut self, limit: u64) -> Self {
self.context_builder.max_cycles(limit);
self
}
/// 메모리 제한 설정
pub fn max_memory(mut self, bytes: usize) -> Self {
self.context_builder.max_memory(bytes);
self
```

```
}
/// 표준 출력 리다이렉션
pub fn stdout<W: Write + Send>(mut self, writer: &'a mut W) -> Self {
self.context_builder.stdout(writer);
self
}
/// 실행 수행
pub fn run(self) -> Result<(SP1PublicValues, ExecutionReport)> {
let context = self.context_builder.build();
let (public_values, _, report) = self.prover.execute(self.elf, &self.stdin, cont
Ok((public_values, report))
}
}
```

CpuProveBuilder - 증명 설정 빌더

```
pub struct CpuProveBuilder<'a> {
prover: &'a CpuProver,
mode: SP1ProofMode,
pk: &'a SP1ProvingKey,
stdin: SP1Stdin,
context_builder: SP1ContextBuilder,
core_opts: SP1CoreOpts,
recursion_opts: SP1CoreOpts,
```

```
mock: bool,
}

impl<'a> CpuProveBuilder<'a> {
/// Core 증명 모드 설정
pub fn core(mut self) -> Self {
self.mode = SP1ProofMode::Core;
self
}
/// 압축 증명 모드 설정
pub fn compressed(mut self) -> Self {
self.mode = SP1ProofMode::Compressed;
self
}
/// PLONK 증명 모드 설정
pub fn plonk(mut self) -> Self {
self.mode = SP1ProofMode::Plonk;
self
}
/// Groth16 증명 모드 설정
pub fn groth16(mut self) -> Self {
self.mode = SP1ProofMode::Groth16;
self
```

```
}
/// 샤드 크기 설정
pub fn shard_size(mut self, size: u32) -> Self {
self.core_opts.shard_size = size;
self
}
/// 증명 생성 실행
pub fn run(self) -> Result<SP1ProofWithPublicValues> {
let context = self.context_builder.build();
self.prover.prove_impl(self.pk, &self.stdin,
SP1ProverOpts {
core_opts: self.core_opts,
recursion_opts: self.recursion_opts
},
context, self.mode)
}
}
```

CPU 증명 생성 상세 프로세스

1. 실행 추적 생성 단계

```
impl CpuProver {
fn execute_and_trace(&self, elf: &[u8], stdin: &SP1Stdin) -> Result<(SP1PublicV
```

```
// 1. 프로그램 로드 및 파싱
let program = Program::from(elf)?;
// 2. 실행기 초기화
let mut executor = Executor::new(program, SP1CoreOpts::default());
executor.executor_mode = ExecutorMode::Trace; // 추적 모드 활성화
// 3. 입력 데이터 설정
executor.write_stdin(stdin);
// 4. 프로그램 실행 (추적 수집)
executor.run()?;
// 5. 결과 추출
let public_values = executor.get_public_values();
let execution_record = *executor.record;
Ok((public_values, execution_record))
}
}
```

2. 샤드 분할 과정

```
impl CpuProver {
fn split_into_shards(&self, mut record: ExecutionRecord) -> Result<Vec<Executi
let mut shards = Vec::new();
let shard_size = self.core_opts.shard_size;
// CPU 이벤트 기준으로 샤드 분할
```

```
let mut current_shard = ExecutionRecord::new(record.program.clone());
let mut current_cycles = 0;
for cpu_event in record.cpu_events.drain(..) {
current_shard.cpu_events.push(cpu_event);
current_cycles += 4; // 기본 명령어 사이클
if current_cycles >= shard_size {
// 현재 샤드 완료, 새 샤드 시작
let completed_shard = std::mem::replace(
&mut current_shard,
ExecutionRecord::new(record.program.clone())
);
shards.push(completed_shard);
current_cycles = 0;
}
}
// 마지막 샤드 추가
if !current_shard.cpu_events.is_empty() {
shards.push(current_shard);
}
// 각 샤드에 관련 이벤트들 분배
self.distribute_events_to_shards(&mut shards, record)?;
Ok(shards)
}
```

```
fn distribute_events_to_shards(&self, shards: &mut [ExecutionRecord], mut reco
// ALU 이벤트 분배
for alu_event in record.add_events.drain(..) {
let shard_idx = self.find_shard_for_pc(shards, alu_event.pc)?;
shards[shard_idx].add_events.push(alu_event);
}
// 메모리 이벤트 분배
for memory_event in record.memory_initialize_events.drain(..) {
let shard_idx = self.find_shard_for_address(shards, memory_event.addr)?;
shards[shard_idx].memory_initialize_events.push(memory_event);
}
// 시스템 콜 이벤트 분배
for syscall_event in record.syscall_events.drain(..) {
let shard_idx = self.find_shard_for_pc(shards, syscall_event.clk)?;
shards[shard_idx].syscall_events.push(syscall_event);
}
Ok(())
}
}
```

3. 개별 샤드 증명 생성

```
impl CpuProver {
```

```
fn prove_shard(&self, pk: &SP1ProvingKey, shard: &ExecutionRecord) -> Result<SP

// 1. 머신 설정

let machine = RiscvAir::machine(BabyBearPoseidon2::new());

// 2. 추적 생성

let traces = machine.generate_traces(shard);

// 3. 제약 조건 평가

let constraints = machine.evaluate_constraints(&traces);

// 4. 다항식 커밋

let commitments = machine.commit_polynomials(&constraints);

// 5. STARK 증명 생성

let proof = machine.prove(&pk.pk, &traces, &commitments)?;

Ok(SP1CoreProofData(proof))

}

}
```

4. 압축 증명 생성 과정

```
impl CpuProver {

fn prove_compressed(&self, pk: &SP1ProvingKey, stdin: &SP1Stdin) -> Result<SP1P

// 1. Core 증명 생성

let core_proof = self.prove_core(pk, stdin)?;

// 2. 재귀적 압축 준비

let compress_machine = CompressAir::machine(BabyBearPoseidon2::new());
```

```
// 3. Core 증명들을 압축 회로 입력으로 변환
let compress_input = self.prepare_compress_input(&core_proof)?;
// 4. 압축 증명 생성
let compressed_proof = compress_machine.compress(&pk.vk, compress_input,
SP1ProverOpts::default())?;
// 5. 결과 래핑
Ok(SP1ProofWithPublicValues::new(
SP1Proof::Compressed(Box::new(compressed_proof)),
core_proof.public_values,
self.version().to_string(),
))
}
fn prepare_compress_input(&self, core_proof: &SP1ProofWithPublicValues) -> Resu
match &core_proof.proof {
SP1Proof::Core(proofs) => {
// Core 증명들을 압축 가능한 형태로 변환
let mut compress_input = CompressInput::new();
for proof in proofs {
// 각 증명의 공개 입력 추출
let public_inputs = self.extract_public_inputs(proof)?;
// 증명 데이터 직렬화
let proof_data = self.serialize_proof(proof)?;
compress_input.add_proof(public_inputs, proof_data);
```

```
}
Ok(compress_input)
}
_ => Err(anyhow::anyhow!("Expected Core proof for compression")),
}
}
}
```

5. SNARK 래핑 과정 (PLONK/Groth16)

```
impl CpuProver {
fn  prove_plonk(&self, pk: &SP1ProvingKey, stdin: &SP1Stdin) -> Result<SP1ProofW
// 1. 압축 증명 생성
let  compressed_proof = self.prove_compressed(pk, stdin)?;
// 2. Shrink 단계 (증명 크기 최적화)
let  shrink_proof = self.shrink_proof(&compressed_proof)?;
// 3. BN254 필드로 래핑
let  wrap_proof = self.wrap_bn254(&shrink_proof)?;
// 4. PLONK 회로 아티팩트 로드
let  plonk_artifacts = self.load_plonk_artifacts(&wrap_proof)?;
// 5. PLONK 증명 생성
let  plonk_proof = self.inner().wrap_plonk_bn254(wrap_proof, &plonk_artifacts);
Ok(SP1ProofWithPublicValues::new(
```

```
SP1Proof::Plonk(plonk_proof),
compressed_proof.public_values,
self.version().to_string(),
))
}
fn shrink_proof(&self, compressed_proof: &SP1ProofWithPublicValues) -> Result<S
match &compressed_proof.proof {
SP1Proof::Compressed(proof) => {
// 압축 증명을 더 작은 형태로 변환
let shrink_machine = ShrinkAir::machine(BabyBearPoseidon2::new());
let shrink_proof = shrink_machine.shrink(proof.as_ref(), SP1ProverOpts::default
Ok(shrink_proof)
}
_ => Err(anyhow::anyhow!("Expected Compressed proof for shrinking")),
}
}
fn wrap_bn254(&self, shrink_proof: &ShrinkProof) -> Result<WrapProof> {
// BN254 타원곡선 필드로 증명 변환
let wrap_machine = WrapAir::machine(Bn254Fr::new());
let wrap_proof = wrap_machine.wrap_bn254(shrink_proof, SP1ProverOpts::default()
Ok(wrap_proof)
}
```

```
}
```

Mock 모드 상세 분석

Mock 모드는 개발 및 테스트 단계에서 실제 증명 생성 없이 빠른 검증을 위해 사용됩니다:

```
impl CpuProver {
fn  mock_prove_impl(&self, pk: &SP1ProvingKey, stdin: &SP1Stdin,
context: SP1Context, mode: SP1ProofMode) -> Result<SP1ProofWithPublicValues> {
// 1. 실제 실행만 수행 (증명 생성 생략)
let (public_values, _, _) = self.prover.execute(&pk.elf, stdin, context)?;
// 2. Mock 증명 생성
let  mock_proof = match  mode {
SP1ProofMode::Core => SP1Proof::Core(vec![]), // 빈 증명
SP1ProofMode::Compressed => SP1Proof::Compressed(Box::new(MockCompressedProof::n
SP1ProofMode::Plonk => SP1Proof::Plonk(MockPlonkProof::new(&public_values)),
SP1ProofMode::Groth16 => SP1Proof::Groth16(MockGroth16Proof::new(&public_values)
};
Ok(SP1ProofWithPublicValues::new(
mock_proof,
public_values,
self.version().to_string(),
))
}
```

```
}
```

성능 최적화 기법

1. 병렬 처리 최적화

```
impl CpuProver {
fn prove_shards_parallel(&self, pk: &SP1ProvingKey, shards: &[ExecutionRecord])
use rayon::prelude::*;
// 샤드별 병렬 증명 생성
let proofs = shards.par_iter()
.map(|shard| self.prove_shard(pk, shard))
.collect::<Result<Vec<_>>>()?;
Ok(proofs)
}
}
```

2. 메모리 사용량 최적화

```
impl CpuProver {
fn optimize_memory_usage(&self, opts: &mut SP1CoreOpts) {
// 사용 가능한 메모리에 따라 샤드 크기 조정
let available_memory = self.get_available_memory();
let optimal_shard_size = self.calculate_optimal_shard_size(available_memory);
```

```
opts.shard_size = optimal_shard_size;

opts.shard_batch_size = std::cmp::min(opts.shard_batch_size,

available_memory / (optimal_shard_size * 2));

}

}
```

사용 예제:

```
use sp1_sdk::{ProverClient, SP1Stdin};


let client = ProverClient::builder().cpu().build();

let elf = include_bytes!("../program.elf");

let stdin = SP1Stdin::new();


// 키 생성

let (pk, vk) = client.setup(elf);


// 다양한 증명 모드

let core_proof = client.prove(&pk, &stdin).core().run().unwrap();

let compressed_proof = client.prove(&pk, &stdin).compressed().run().unwrap();

let plonk_proof = client.prove(&pk, &stdin).plonk().run().unwrap();

let groth16_proof = client.prove(&pk, &stdin).groth16().run().unwrap();
```

4. CudaProver - GPU 가속 증명 생성기

NVIDIA GPU를 활용하여 증명 생성을 가속화하는 프로버입니다.

구조 및 특징

```
pub struct CudaProver {
pub(crate) cpu_prover: SP1Prover<CpuProverComponents>,
pub(crate) cuda_prover: SP1CudaProver,
}

impl CudaProver {
pub fn new(prover: SP1Prover, moongate_server: MoongateServer) -> Self { /* ..
// CPU와 동일한 실행 인터페이스
pub fn execute<'a>(&'a self, elf: &'a [u8], stdin: &SP1Stdin) -> CpuExecuteBui
// CUDA 특화 증명 빌더
pub fn prove<'a>(&'a self, pk: &'a SP1ProvingKey, stdin: &'a SP1Stdin) -> Cuda
// 사이클 수와 함께 증명 반환
pub fn prove_with_cycles(&self, pk: &SP1ProvingKey, stdin: &SP1Stdin, kind: SP
}
```

핵심 특징:

- **하이브리드 아키텍처**: CPU로 실행, GPU로 증명 생성
- **Moongate 서버**: CUDA 연산을 관리하는 백그라운드 서버

- **성능 최적화**: 대규모 프로그램에 대한 빠른 증명 생성
- **사이클 추적**: 증명과 함께 실행 사이클 수 반환

GPU 가속 프로세스:

1. **Core Proof**: GPU에서 병렬 STARK 증명 생성
2. **Compression**: GPU 기반 증명 압축
3. **Shrinkage**: GPU에서 증명 크기 최적화
4. **Wrapping**: CPU에서 최종 SNARK 래핑

사용 예제:

```
use sp1_sdk::{ProverClient, SP1Stdin};


let client = ProverClient::builder().cuda().build();
let elf = include_bytes!("../large_program.elf");
let stdin = SP1Stdin::new();


let (pk, vk) = client.setup(elf);


// 사이클 수와 함께 증명 생성
let (proof, cycles) = client.prove_with_cycles(&pk, &stdin, SP1ProofMode::Compre
println!("Generated proof with {} cycles", cycles);
```

5. NetworkProver - 네트워크 기반 증명 생성기

원격 서버에서 증명을 생성하는 클라우드 기반 프로버입니다.

구조 및 특징

```rust
pub struct NetworkProver {
client: NetworkClient,
private_key: Option<String>,
signer: Option<Box<dyn NetworkSigner>>,
}

impl NetworkProver {
pub fn new(private_key: &str) -> Self { /* ... */ }
pub fn with_signer(signer: Box<dyn NetworkSigner>) -> Self { /* ... */ }
}
```

주요 특징:

- **클라우드 증명**: 로컬 리소스 없이 증명 생성
- **확장성**: 대규모 프로그램에 대한 무제한 컴퓨팅 파워
- **인증 시스템**: 개인 키 또는 커스텀 서명자를 통한 인증
- **비용 효율성**: 필요할 때만 컴퓨팅 리소스 사용

네트워크 설정:

```rust
use sp1_sdk::{ProverClient, SP1Stdin};

// 개인 키를 통한 인증
```

```
std::env::set_var("NETWORK_PRIVATE_KEY", "your_private_key");
let client = ProverClient::builder().network().build();


// 또는 직접 설정
let client = ProverClient::builder()
.network()
.private_key("your_private_key")
.rpc_url("https://rpc.mainnet.succinct.xyz")
.build();
```

6. EnvProver - 환경 기반 프로버

환경 변수를 통해 자동으로 적절한 프로버를 선택하는 어댑터입니다.

환경 변수 설정

```
# CPU 프로버 사용
export SP1_PROVER=cpu


# CUDA 프로버 사용
export SP1_PROVER=cuda


# 네트워크 프로버 사용
export SP1_PROVER=network
```

```
export NETWORK_PRIVATE_KEY=your_private_key
export NETWORK_RPC_URL=https://rpc.mainnet.succinct.xyz

# Mock 모드 사용
export SP1_PROVER=mock
```

사용 예제:

```
use sp1_sdk::{ProverClient, SP1Stdin};

// 환경 변수에 따라 자동으로 프로버 선택
let client = ProverClient::from_env();

let elf = include_bytes!("../program.elf");
let stdin = SP1Stdin::new();
let (pk, vk) = client.setup(elf);
let proof = client.prove(&pk, &stdin).run().unwrap();
```

API 사용 패턴

1. 기본 워크플로우

```
use sp1_sdk::{ProverClient, SP1Stdin, Prover};
```

```rust
// 1. 클라이언트 생성
let client = ProverClient::builder().cpu().build();

// 2. 프로그램 로드
let elf = include_bytes!("../program.elf");

// 3. 입력 데이터 준비
let mut stdin = SP1Stdin::new();
stdin.write(&42u32);

// 4. 키 생성 (한 번만 수행)
let (pk, vk) = client.setup(elf);

// 5. 프로그램 실행 (선택사항)
let (public_values, execution_report) = client.execute(elf, &stdin).run()?;

// 6. 증명 생성
let proof = client.prove(&pk, &stdin).compressed().run()?;

// 7. 증명 검증
client.verify(&proof, &vk)?;
```

2. 고급 설정 옵션

```rust
use sp1_sdk::{ProverClient, SP1Stdin, SP1Context};


let client = ProverClient::builder().cpu().build();
let elf = include_bytes!("../program.elf");
let stdin = SP1Stdin::new();
let (pk, vk) = client.setup(elf);


// 실행 옵션 설정
let (public_values, report) = client
.execute(elf, &stdin)
.cycle_limit(1_000_000) // 최대 사이클 수 제한
.max_memory(1024 * 1024) // 최대 메모리 사용량
.run()?;


// 증명 옵션 설정
let proof = client
.prove(&pk, &stdin)
.compressed() // 압축 모드
.shard_size(1 << 22) // 샤드 크기 설정
.run()?;
```

3. 에러 처리

```rust
use sp1_sdk::{ProverClient, SP1VerificationError};


let  client = ProverClient::builder().cpu().build();

// ... 설정 코드 ...


match  client.verify(&proof, &vk) {

Ok(()) => println!("증명 검증 성공"),

Err(SP1VerificationError::InvalidPublicValues) => {

println!("공개 값이 유효하지 않습니다");

},

Err(SP1VerificationError::VersionMismatch(version)) => {

println!("버전 불일치: {}", version);

},

Err(SP1VerificationError::Core(err)) => {

println!("Core 증명 검증 실패: {}", err);

},

Err(e) => println!("기타 오류: {}", e),

}
```

성능 고려사항

1. 프로버 선택 가이드

| 프로버 타입 | 적합한 사용 사례 | 장점 | 단점 |

|---|---|---|---|

| CpuProver | 소규모 프로그램, 개발/테스트 | 설정 간단, 로컬 실행 | 느린 속도 |

| CudaProver | 중대규모 프로그램, 로컬 GPU 보유 | 빠른 속도, 로컬 제어 | GPU 하드웨어 필요 |

| NetworkProver | 대규모 프로그램, 클라우드 환경 | 무제한 성능, 하드웨어 불필요 | 네트워크 의존, 비용 |

| Mock | 개발/테스트 | 즉시 실행 | 실제 증명 없음 |

2. 메모리 및 성능 최적화

```
use sp1_sdk::{ProverClient, SP1Context};

let client = ProverClient::builder().cpu().build();

// 메모리 효율적인 설정
let context = SP1Context::builder()
.shard_size(1 << 20) // 작은 샤드 크기로 메모리 사용량 감소
.max_cycles(10_000_000) // 사이클 수 제한
.build();

// 성능 우선 설정
let context = SP1Context::builder()
.shard_size(1 << 24) // 큰 샤드 크기로 처리량 증가
.build();
```

요약

SP1 SDK는 다양한 환경과 요구사항에 맞는 유연한 증명 생성 솔루션을 제공합니다. 통일된 API를 통해 CPU, GPU, 네트워크 기반 증명 생성을 지원하며, 빌더 패턴을 통한 세밀한 설정이 가능합니다. 개발자는 프로젝트의 규모와 환경에 따라 적절한 프로버를 선택하여 최적의 성능을 얻을 수 있습니다.